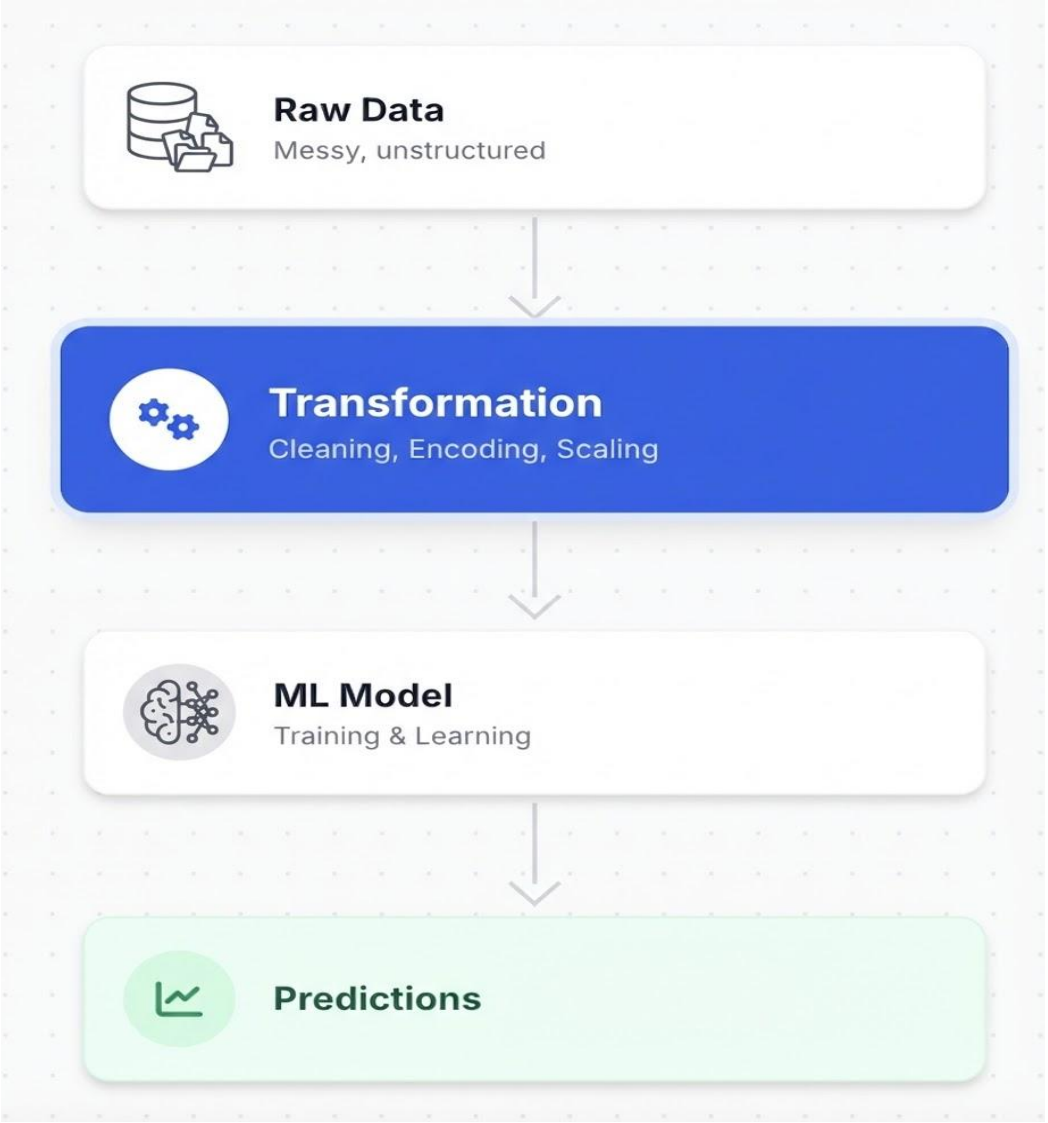
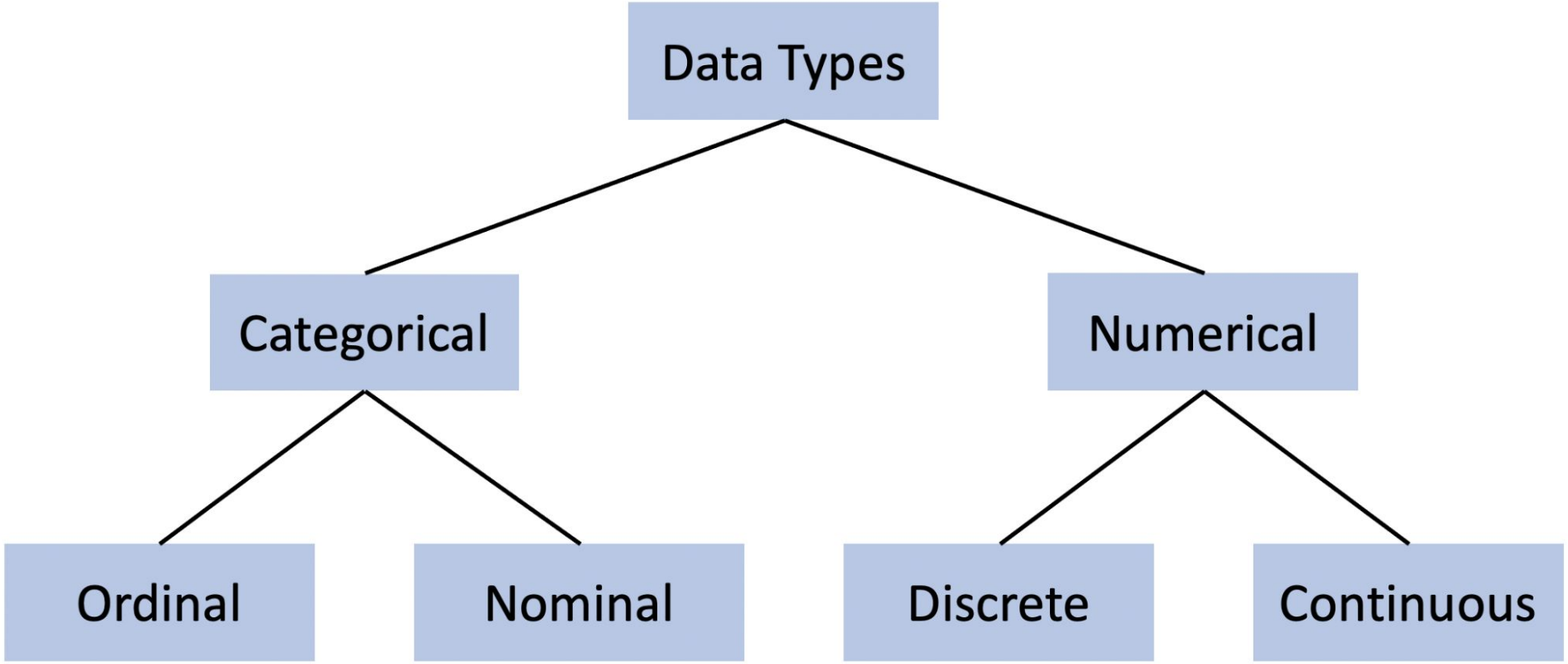
Data Transformation

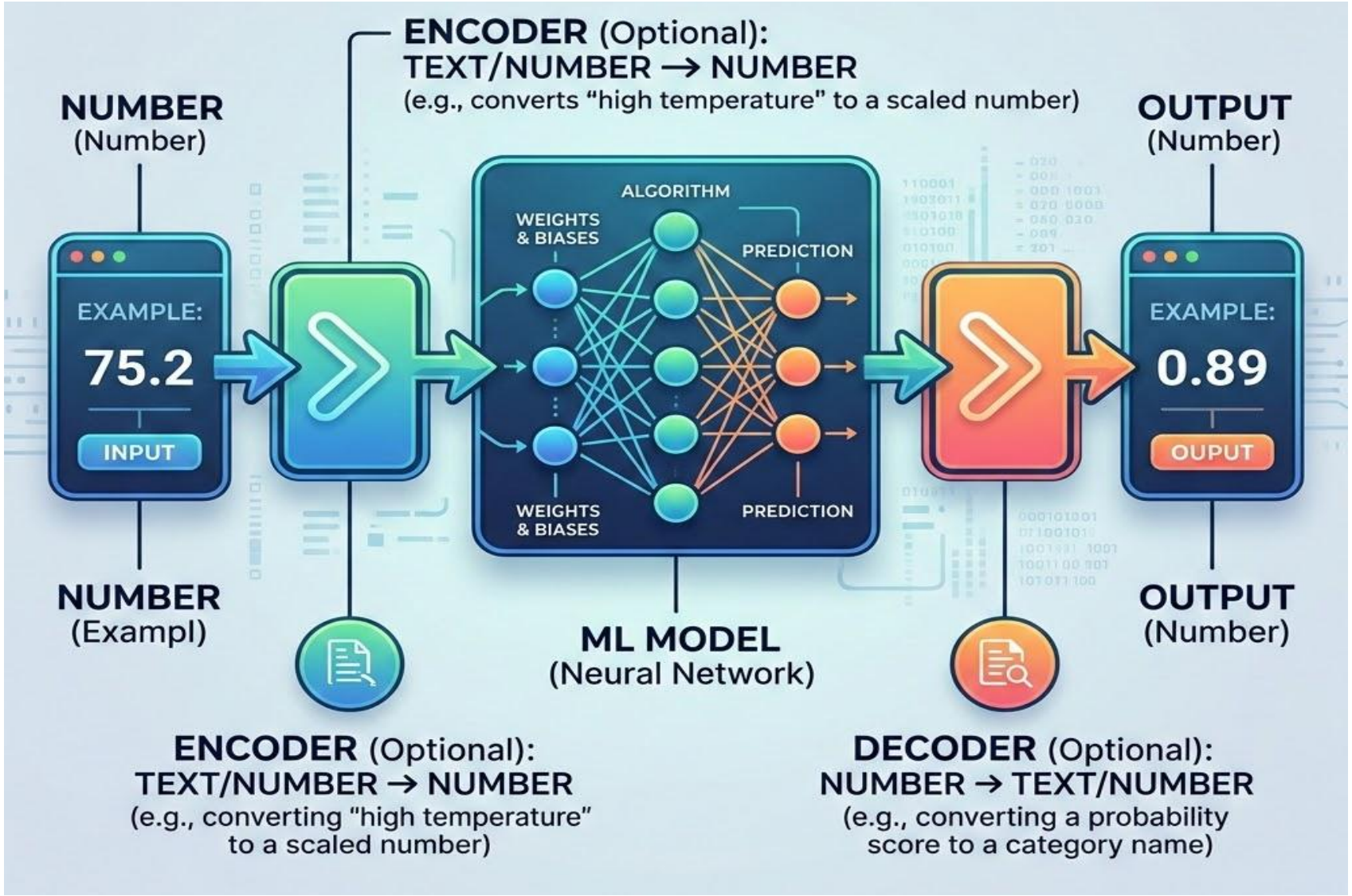
for Machine Learning



Types of Data



ML Model Workflow



Machines Only Understand Numbers

Categorical data must be converted before training

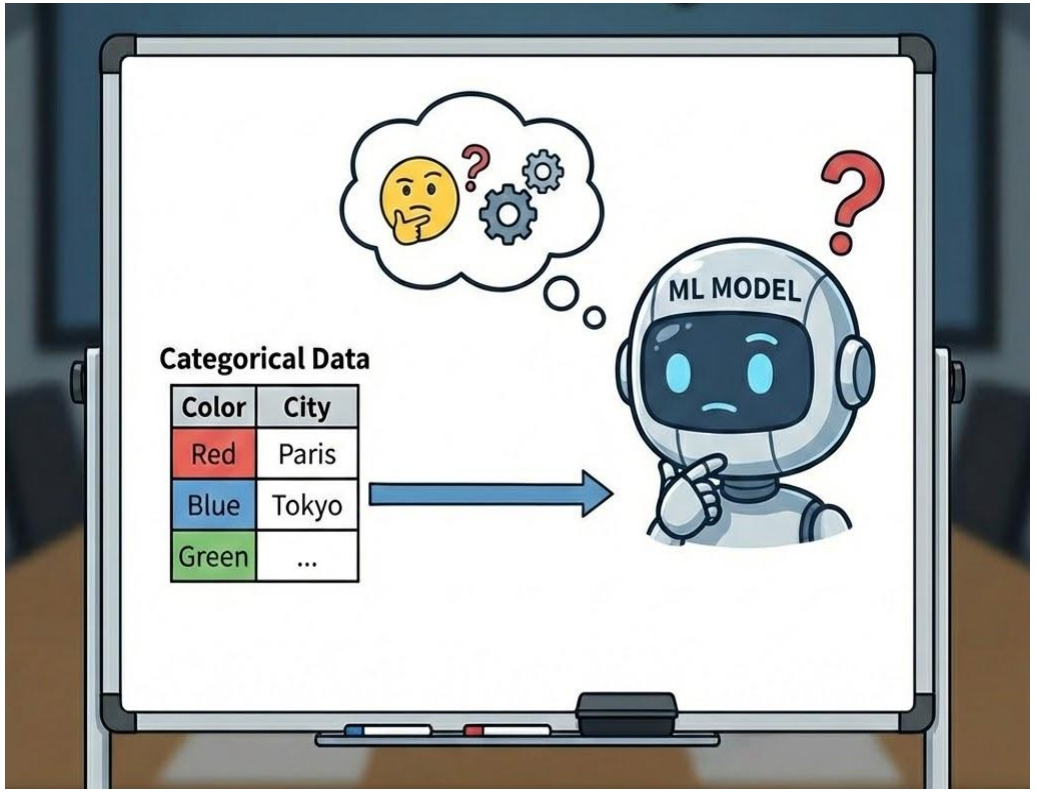
Every ML algorithm performs mathematical operations during training: multiplications, distances, dot products. None of that works on text.

What the data looks like

"Dhaka", "Chittagong", "Sylhet"

What the model needs

0, 1, 2 (or some numeric form)



Label Encoding

Assigns a unique integer to each category.

City	Encoded
Dhaka	0
Chittagong	1
Sylhet	2
Chittagong	1

Problem: The model now treats Sylhet (2) as greater than Dhaka (0). For a city column that ordering is meaningless and will mislead any model that computes distances or dot products.

Ordinal Encoding

Same mechanic as label encoding, but you explicitly define the order mapping yourself.

Education Level	Encoded
High School	1
Bachelor	2
Master	3
PhD	4
Bachelor	2

The order here is real: PhD > Master > Bachelor > High School. You define the mapping. The model learns a relationship that actually exists in the data.

One-Hot Encoding

No ordering. Each category becomes its own binary column.

Creates one new column per category. A row gets a 1 in its column and 0 everywhere else.

City	City_Dhaka	City_Chittagong	City_Sylhet
Dhaka	1	0	0
Chittagong	0	1	0
Sylhet	0	0	1
Chittagong	0	1	0
Sylhet	0	0	1

No false ordering. But a column with 500 unique cities produces 500 new columns.

Binary Encoding

How It Works:

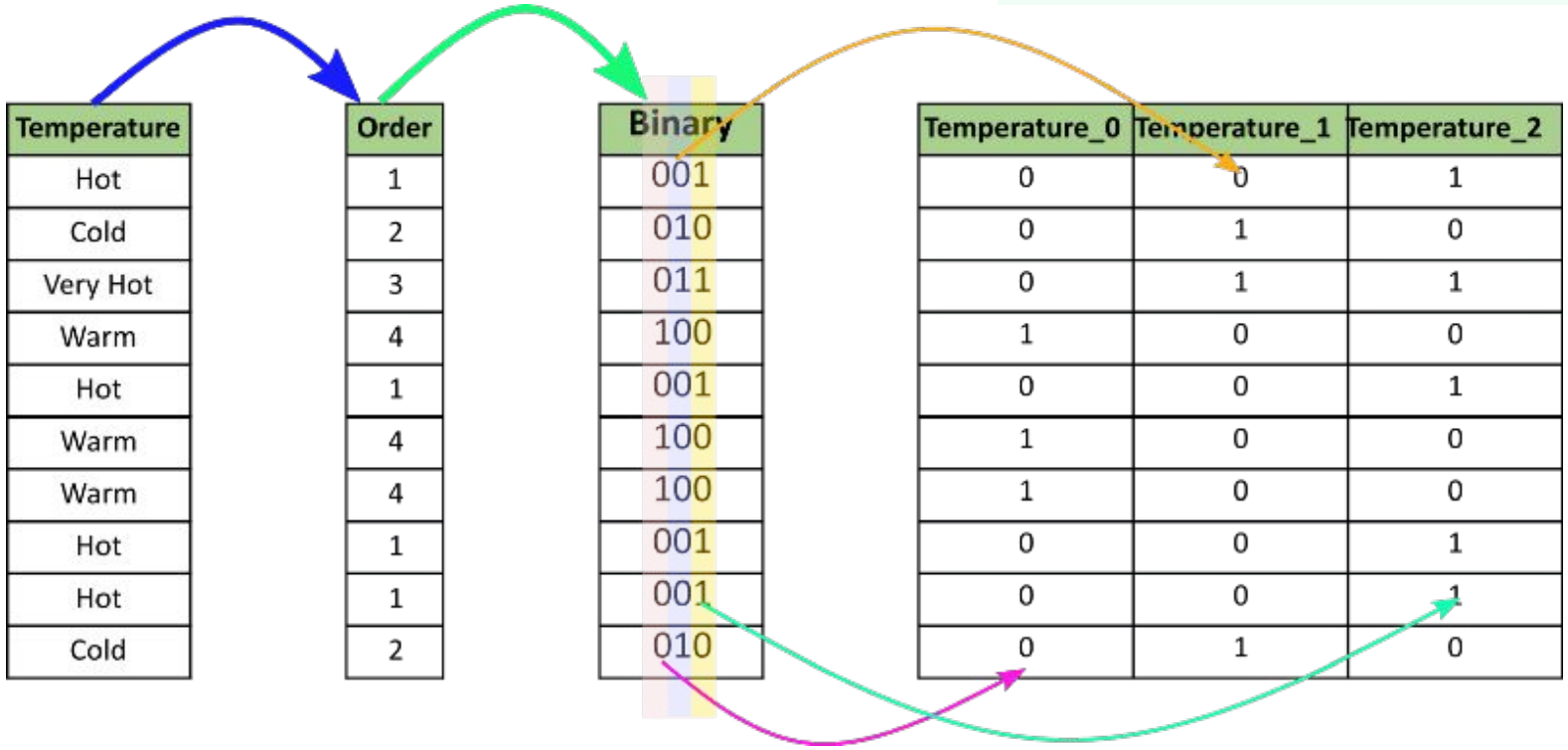
1. Assign integers to categories (like Label Encoding).
2. Convert integers to binary code.
3. Split binary digits into separate columns.

Dimensionality Advantage

Drastically fewer columns than One-Hot Encoding.

Columns ≈ $\log_2$(N categories)

Example: 100 cities → 7 columns (Binary) vs 100 columns (One-Hot)



Temperature	Order	Binary
Hot	1	001
Cold	2	010
Very Hot	3	011
Warm	4	100
Hot	1	001
Warm	4	100
Warm	4	100
Hot	1	001
Hot	1	001
Cold	2	010

Temperature_0	Temperature_1	Temperature_2
0	0	1
0	1	0
0	1	1
1	0	0
0	0	1
1	0	0
1	0	0
0	0	1
0	0	1
0	1	0

Scale Difference Breaks Distance-Based Calculations

Suppose an algorithm needs to calculate the distance between two data points to make a decision. Consider two features in the same dataset:

Feature	Range
Age	20 to 60
Salary	30,000 to 200,000

Distance is almost entirely determined by Salary because its numbers are thousands of times larger. Age contributes almost nothing, not because Age is unimportant, but simply because its scale is smaller.

Scaling fixes this by putting all features on a comparable range.

MinMaxScaler (Normalization)

Formula: `(x − min) / (max − min)` Result: All values between 0 and 1

Original	Scaled
20	0.00
30	0.25
40	0.50
50	0.75
60	1.00

- Use when: model needs bounded inputs (neural networks), or data is not normally distributed
- Weakness: one extreme outlier compresses all other values near zero. A salary of 10,000,000 where most are 30K-200K flattens everything else.

StandardScaler (Z-Score Normalization)

Formula: `(x - mean) / std;` Result: Mean = 0, Std Dev = 1

Original	Scaled
20	-1.41
30	-0.71
40	0.00
50	0.71
60	1.41

- **Use when:** data is roughly normally distributed, features have different units
- **Weakness:** outliers shift the mean and std, so extreme values still affect the scaling

RobustScaler

Formula: `(x − median) / IQR` IQR = 75th percentile − 25th percentile

Why median and IQR hold up when outliers exist

- Median: the middle value of sorted data. An extreme outlier does not move it.
- IQR: the spread of the middle 50% of data. Extreme values in the top or bottom 25% are excluded.
- Mean: shifts toward every outlier. One salary of 10,000,000 drags the mean far from typical values.

- Use when: Data contains outliers you cannot or do not want to remove, and you still need to scale for a distance-based or gradient-based model.
- Not suitable when: The downstream model requires inputs strictly in the [0, 1] range.

Three Sets, Three Jobs

Training Set	The model learns from this data. It sees features and labels, adjusts its parameters, and builds its understanding of patterns. This is where all learning happens.
Validation Set	Used during development to compare models and tune hyperparameters. You are allowed to look at validation performance repeatedly while you iterate.
Test Set	Evaluated exactly once, at the very end. It answers the only honest question: how does this model perform on data it has never influenced in any way?

Looking at Test Data Early Poisons Your Results

This mistake produces models that fail in production

Every time you use test data to make a decision, select a model, tune a threshold, remove an outlier; you are leaking information about the test set into your training process.

01 Model is optimized for that specific test set, not for genuinely unseen data.

02 Reported accuracy is inflated. The number looks good. Real-world performance is lower.

03 You cannot recover a clean estimate. Once the test set influenced any decision, the number it produces is no longer honest.

Rule: The test set is used once. After that, the number it produces is your final answer.

Common Split Ratios and Stratification

The ratio matters less than doing it correctly

Setup	Train	Validation	Test
With validation set	70%	15%	15%
Without validation set	80%	--	20%

STRATIFIED SPLITTING

- When splitting a classification dataset, ensure each split has the same class proportions as the original.
- Without stratification, a random split on an imbalanced dataset might put 95% of the minority class into training and leave almost none in test. The test result becomes meaningless.

Data Leakage

Data leakage happens when information from outside the training set is used to build the model.

Validation and test scores look excellent. Actual performance is significantly lower.

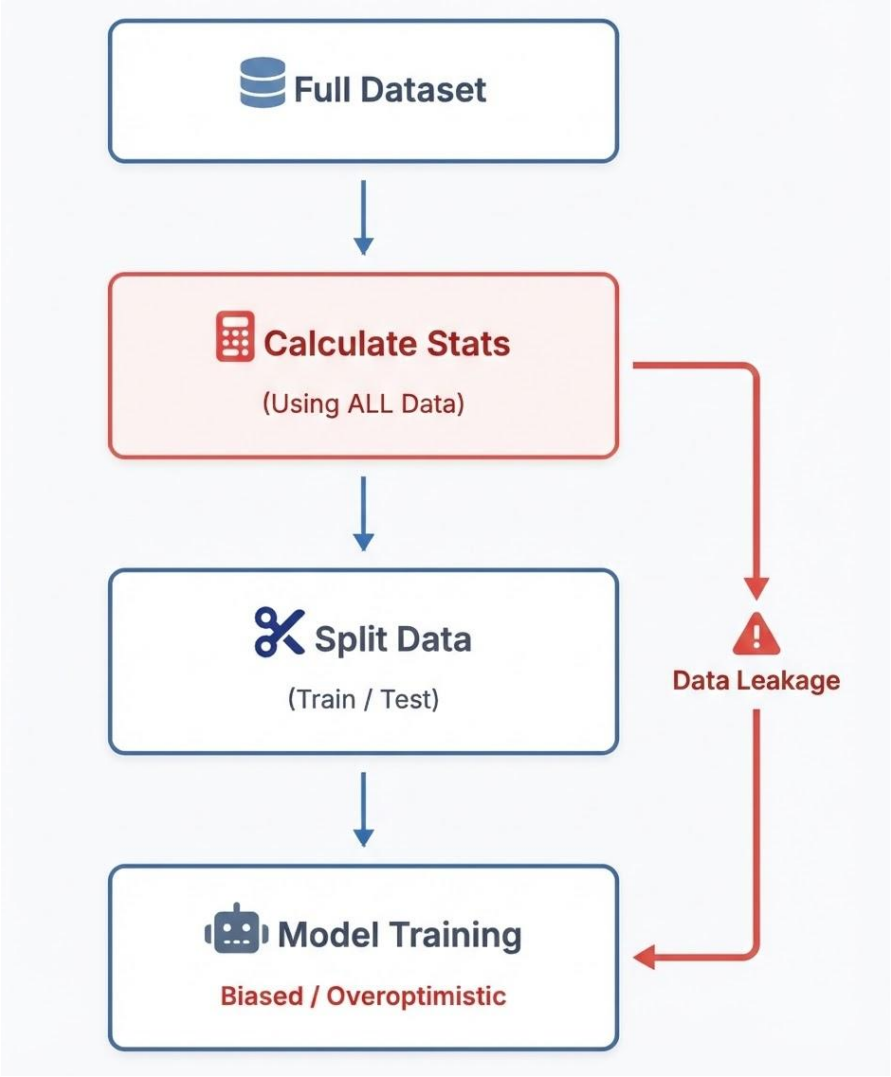
Leakage is invisible in your numbers. The only way to catch it is to think carefully about the order of every operation in your pipeline.

Fitting the scaler on the full dataset before splitting

The scaler learns the mean and standard deviation of test data. That information then influences how training data is scaled.

Filling missing values with the overall mean before splitting

The mean includes test set values. The test set has effectively told the model something about itself before training.



99% Accuracy That Tells You Nothing

A credit card fraud detection dataset:

Class	Samples
Not fraud	99,800
Fraud	200

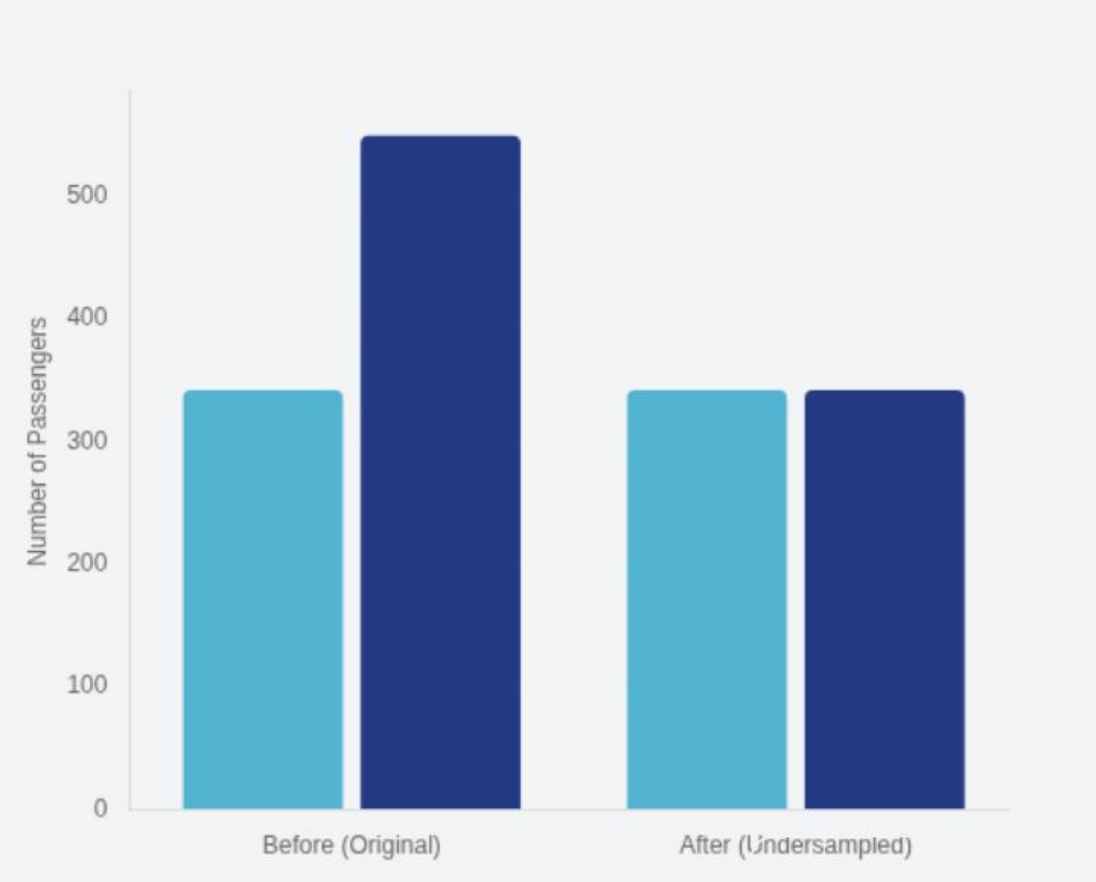
A model that predicts "not fraud" for every transaction achieves 99.8% accuracy.

- It catches zero fraud cases. It is completely useless.
- Accuracy is the wrong metric when classes are imbalanced.
- Training without handling imbalance produces exactly this kind of model.

Handling Imbalance: Random Undersampling

How it works:

1. Keep ALL minority class samples.
2. Randomly select SAME number from majority class.
3. Discard remaining majority samples.



SMOTE: Synthetic Minority Oversampling Technique

Create new minority samples instead of duplicating existing ones.

How it works:

1. For each minority class sample, find its nearest neighbors (also minority class).
2. Pick one neighbor at random.
3. Create a new synthetic sample at a random point along the line between the two.

Class	Before SMOTE	After SMOTE
Not fraud	99,800	99,800 (unchanged)
Fraud	200	99,800 (original + synthetic)

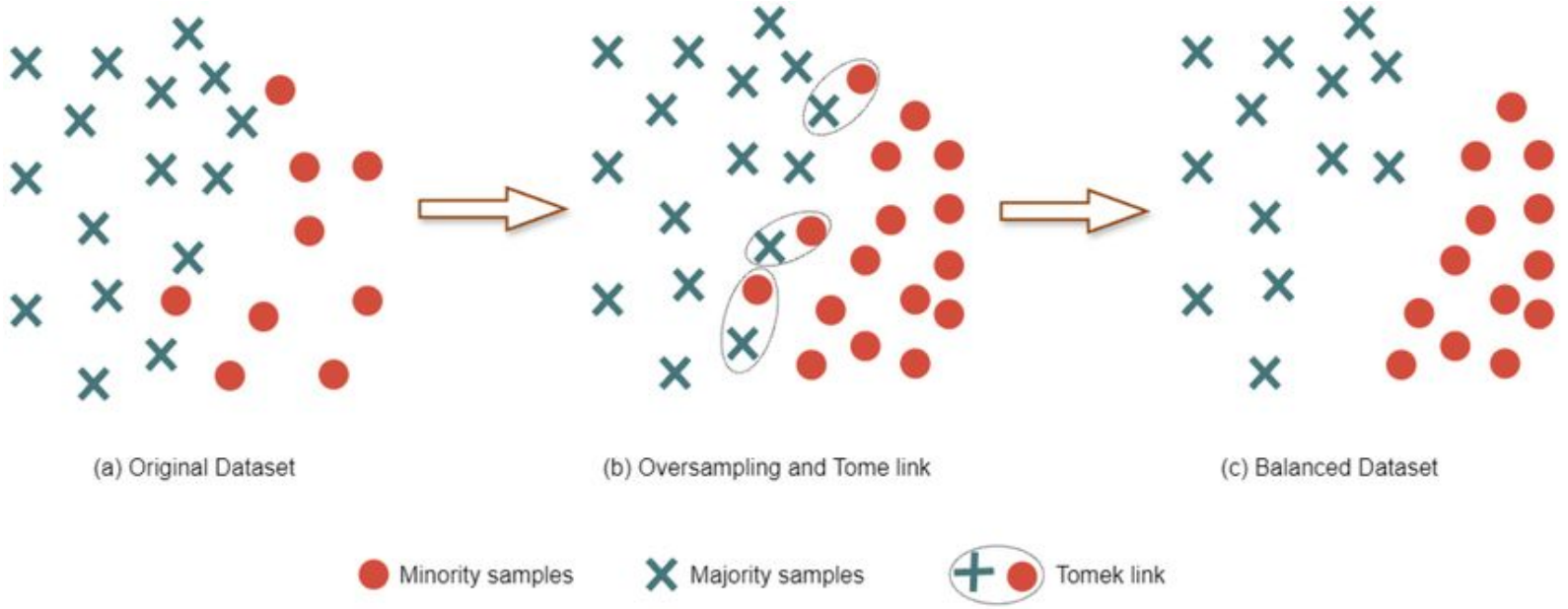
Limitation: SMOTE generates samples without checking class boundaries. Some synthetic samples may land in regions that actually belong to the majority class.

SMOTETomek: Oversample, Then Clean the Boundary

SMOTE plus removal of ambiguous borderline samples

How it works:

1. Oversampling: Generate synthetic minority samples using SMOTE to balance the class distribution.
2. Cleaning: Identify Tomek Links, which are pairs of nearest neighbors from opposite classes.
3. Boundary Refinement: Remove these ambiguous borderline pairs to increase the separation between classes and reduce noise.



Class Weights: Reweight the Loss, Not the Data

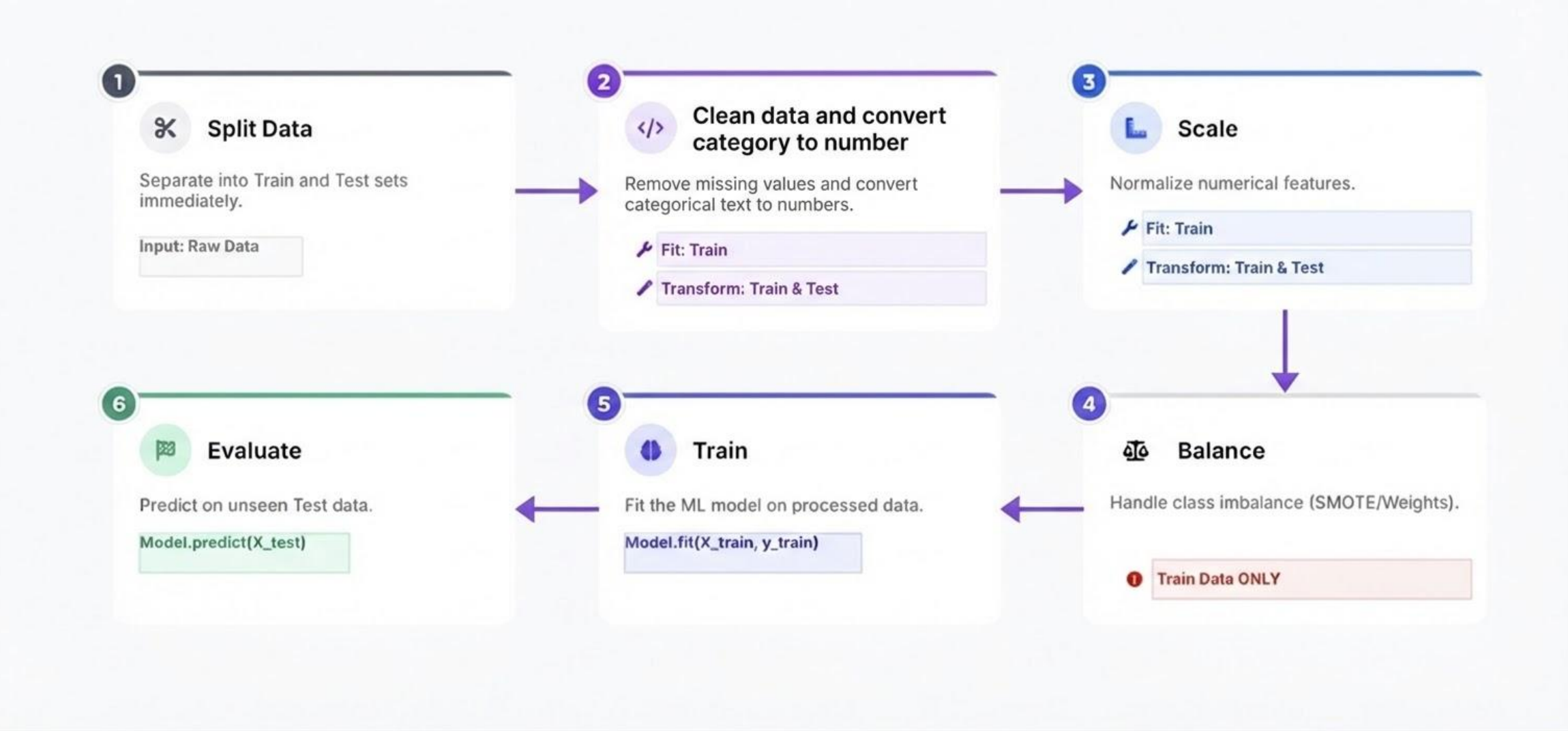
Instead of changing the dataset, tell the model that mistakes on the minority class cost more.

Class	Weight	Effect
Majority (Not fraud)	1.0	Normal penalty for mistakes
Minority (Fraud)	10.0	10x penalty for mistakes

Advantages:

- No data is added or removed
- Training is faster than SMOTE
- No risk of synthetic data falling in the wrong class region

Complete Pipeline



Thank you!